Overall Approach to Project Completion_01

[MEMORY BANK: ACTIVE]

I'll analyze the project structure and implementation guides to provide strategic recommendations for completing Reachy_08.4.2.

Let me first read the memory bank core files, then review the implementation guides and n8n documentation.

Strategic Analysis: Completing Reachy_08.4.2

Based on my analysis of the implementation guides, n8n workflows, and current project status, I'll provide a comprehensive strategic assessment.

Current Project Status

Completed (60%):

- ✅ Phase 1: Web UI & Foundation (100%)
- ✅ Phase 2: ML Pipeline (100%)
- ✅ Phase 3: Edge Deployment (100%)
- ✅ n8n Workflows: 9 production-ready JSON files exist

Remaining (40%):

- 🟡 Phase 4: n8n Orchestration (workflows exist but not deployed/tested)
- ⚪ Phase 5: Production Hardening (not started)

Possible Approaches

Approach 1: Agent-by-Agent Sequential Implementation

⭐ **RECOMMENDED**

Strategy: Complete each of the 9 agents sequentially with full integration testing

Process:

1. Deploy Agent 1 (Ingest) → Test with Media Mover API → Verify DB writes

2. Deploy Agent 2 (Labeling) → Test with Web UI → Verify class balance tracking
3. Deploy Agent 3 (Promotion) → Test dry-run/approval → Verify manifest rebuild
4. Deploy Agent 4 (Reconciler) → Test filesystem scan → Verify drift detection
5. Deploy Agent 5 (Training) → Test TAO integration → Verify MLflow tracking
6. Deploy Agent 6 (Evaluation) → Test Jetson inference → Verify Gate B
7. Deploy Agent 7 (Deployment) → Test shadow→canary→rollout → Verify health checks
8. Deploy Agent 8 (Privacy) → Test TTL/GDPR → Verify audit logs
9. Deploy Agent 9 (Observability) → Test metrics collection → Verify alerting

Pros:

- ✅ **Incremental validation**: Each agent fully tested before moving to next
- ✅ **Early value delivery**: Core agents (1-3) provide immediate workflow automation
- ✅ **Lower risk**: Problems isolated to one agent at a time
- ✅ **Clear milestones**: Each agent completion is a deliverable
- ✅ **Matches existing code**: Phases 1-3 APIs already support these workflows

Cons:

- ⚠️ Longer total timeline (but more predictable)
- ⚠️ Some agents depend on others (must follow dependency order)

Time Estimate: 12-15 days (1-2 days per agent)

Approach 2: Layer-by-Layer Implementation

Strategy: Complete all core agents first, then advanced agents, then monitoring

Process:

1. **Week 1**: Core data flow (Agents 1-3: Ingest → Labeling → Promotion)
2. **Week 2**: ML automation (Agents 4-6: Reconciler → Training → Evaluation)
3. **Week 3**: Deployment & ops (Agents 7-9: Deployment → Privacy → Observability)

Pros:

- ✅ Parallel testing within each layer
- ✅ Logical grouping by domain

Cons:

- ⚠️ **No end-to-end validation until Week 3**
- ⚠️ Higher risk of integration issues discovered late
- ⚠️ May need rework if cross-layer dependencies missed

Time Estimate: 15-18 days

Approach 3: Vertical Slice Implementation

Strategy: Build one complete end-to-end flow first, then expand

Process:

1. **Slice 1**: Upload → Ingest → Label → Promote (manual trigger)
2. **Slice 2**: Add Training → Evaluation → Deployment (full pipeline)
3. **Slice 3**: Add Reconciler, Privacy, Observability (operations)

Pros:

- ✅ Early end-to-end validation
- ✅ Demonstrates complete value quickly

Cons:

- ⚠️ **Integration complexity**: Must coordinate multiple agents simultaneously
- ⚠️ Harder to debug when issues span agents
- ⚠️ Your existing APIs may not be fully tested for coordinated workflows

Time Estimate: 14-17 days

Approach 4: Big Bang Deployment

Strategy: Deploy all 9 workflows simultaneously and debug

Process:

1. Import all 9 JSON files to n8n
2. Configure all credentials
3. Activate all workflows
4. Debug integration issues

Pros:

- ✅ Fastest initial deployment (2-3 hours)

Cons:

- ❌ **High risk**: Complex debugging across 9 agents
- ❌ No isolation of problems
- ❌ Difficult to attribute failures
- ❌ **NOT RECOMMENDED for production systems**

Time Estimate: 2-3 hours deployment + 20-30 days debugging

🎯 RECOMMENDED APPROACH: Agent-by-Agent (Approach 1)

Why This is Optimal

1. **Your code is production-ready**: Phases 1-3 already implemented and tested (90%+ test pass rate)
2. **Workflows are designed**: All 9 n8n JSONs exist and validated
3. **Clear dependencies**: Agent order naturally follows data flow
4. **Risk mitigation**: Each agent can be validated before proceeding
5. **Incremental value**: Early agents provide immediate automation benefits

Dependency-Ordered Sequence

```
Independent (can start immediately):
├─ Agent 1: Ingest       [Day 1-2]
```

```
├─ Agent 4: Reconciler    [Day 7-8]   (independent background process)
└─ Agent 9: Observability  [Day 13-14] (independent monitoring)

Sequential (must follow order):
├─ Agent 2: Labeling      [Day 3-4]   (depends on Agent 1)
├─ Agent 3: Promotion     [Day 5-6]   (depends on Agent 2)
├─ Agent 5: Training      [Day 9-10]  (depends on Agent 3)
├─ Agent 6: Evaluation    [Day 11-12] (depends on Agent 5)
├─ Agent 7: Deployment    [Day 12-13] (depends on Agent 6)
└─ Agent 8: Privacy       [Day 14-15] (can run anytime, placed at end)
```

📋 COMPREHENSIVE ACTION PLAN

Phase 4A: Core Agent Deployment (Days 1-8)

Day 1-2: Agent 1 - Ingest Agent

Setup:

```
# 1. Install n8n (if not already)
npm install -g n8n
# Or use Docker
docker run -d --name n8n -p 5678:5678 \
  -v ~/.n8n:/home/node/.n8n \
  -e N8N_METRICS=true \
  docker.n8n.io/n8nio/n8n

# 2. Import workflow
n8n import:workflow --input=n8n/workflows/01_ingest_agent.json
```

Configure:

- PostgreSQL credential (reachy_local database)
- HTTP auth for Media Mover API (http://10.0.4.130:8081/api)
- Set INGEST_TOKEN environment variable

Test:

```
# Test webhook
curl -X POST http://localhost:5678/webhook/video_gen_hook \
  -H "X-INGEST-KEY: ${INGEST_TOKEN}" \
  -H "Content-Type: application/json" \
  -d '{
    "source_url": "https://example.com/test.mp4",
    "label": "happy"
  }'

# Verify in database
psql reachy_local -c "SELECT video_id, filename, sha256, split FROM video ORDER BY created_at DESC LIMIT 5;"
```

Success Criteria:

- ✅ Video pulled by Media Mover
- ✅ SHA256 hash computed and stored
- ✅ Thumbnail generated
- ✅ Database record created with split='temp'
- ✅ No duplicate inserts on retry

Time: 4-6 hours (2 hours setup, 2-4 hours testing)

Day 3-4: Agent 2 - Labeling Agent

Import & Configure:

```
n8n import:workflow --input=n8n/workflows/02_labeling_agent.json
```

Test via Web UI:

1. Open Streamlit UI (http://localhost:8501)
2. Navigate to Video Management page
3. Label a video as "happy"
4. Choose action: "promote_train"

5. Verify webhook triggers Agent 2

Verify:

```
-- Check label_event table
SELECT * FROM label_event ORDER BY created_at DESC LIMIT 5;

-- Check video.label updated
SELECT video_id, filename, label, split FROM video WHERE label IS NOT NULL;

-- Check class balance
SELECT label, split, COUNT(*) FROM video GROUP BY label, split;
```

Success Criteria:

- ✅ Label persisted to database
- ✅ Class balance tracked correctly
- ✅ Web UI receives updated stats
- ✅ Idempotency prevents duplicate labels

Time: 4-6 hours

Day 5-6: Agent 3 - Promotion/Curation Agent

Import & Configure:

```
n8n import:workflow --input=n8n/workflows/03_promotion_agent.json
```

Configure approval webhook:

- Set up Slack webhook (or email) for approval notifications
- Configure dry-run endpoint

Test Flow:

```
# Trigger promotion (dry-run)
curl -X POST http://localhost:5678/webhook/promotion/v1 \
  -H "Content-Type: application/json" \
```

```
 -d '{
   "video_id": "<UUID>",
   "label": "happy",
   "target": "train"
 }'

# Review dry-run output
# Approve via webhook or Slack
# Verify file moved from temp/ to train/
ls -lh /media/project_data/reachy_emotion/videos/train/

# Verify manifest rebuilt
cat /media/project_data/reachy_emotion/videos/manifests/train_manifest.jsonl
```

Success Criteria:

- ✅ Dry-run preview shows planned changes
- ✅ Approval gate blocks until confirmed
- ✅ Atomic file move (temp → train)
- ✅ Database split field updated
- ✅ Manifest rebuilt correctly
- ✅ Rollback works if issues detected

Time: 6-8 hours (includes approval flow testing)

Day 7-8: Agent 4 - Reconciler/Audit Agent

Import & Configure:

```
n8n import:workflow --input=n8n/workflows/04_reconciler_agent.json
```

Configure SSH:

- Add SSH credential for Ubuntu 1 (10.0.4.130)
- Test SSH connection manually

Manual Trigger Test:

```
# Trigger reconciler
curl -X GET http://localhost:5678/webhook/reconciler/audit

# Wait for completion (check email or n8n execution log)
```

Review Report:

- Check email for reconciler report
- Review orphans, missing files, mismatches
- Verify database updated if safe_fix=true

Schedule:

- Verify cron trigger set for 02:15 AM daily

Success Criteria:

- ✅ SSH filesystem enumeration works
- ✅ Database comparison accurate
- ✅ Report identifies drift correctly
- ✅ Email notification received
- ✅ safe_fix updates DB without deleting files

Time: 4-6 hours

Phase 4B: ML Automation (Days 9-12)

Day 9-10: Agent 5 - Training Orchestrator

Import & Configure:

```
n8n import:workflow --input=n8n/workflows/05_training_orchestrator.json
```

Prerequisites:

- TAO environment running (Phase 2 complete)

- MLflow tracking configured
- Training dataset balanced (≥50 per class)

Test:

```
# Trigger training
curl -X POST http://localhost:5678/webhook/agent/training/start \
  -H "Content-Type: application/json" \
  -d '{
    "dataset_hash": "abc123",
    "manifest_path": "/media/.../train_manifest.jsonl",
    "correlation_id": "test-001"
  }'
```

Monitor:

```
# Watch n8n execution (Wait node will poll for completion)
# Check MLflow UI
mlflow ui --backend-store-uri file:///workspace/mlruns

# Check TAO logs
tail -f /workspace/experiments/<run_id>/train.log
```

Success Criteria:

- ✅ MLflow run created with tags
- ✅ TAO training launches successfully
- ✅ Wait node polls without blocking n8n
- ✅ Gate A validation enforced (F1 ≥ 0.84)
- ✅ TensorRT export on pass
- ✅ training.completed event emitted

Time: 8-10 hours (includes training run ~2-4 hours)

Challenges:

- TAO may fail if dataset imbalanced

- GPU memory issues if other processes running
- Wait node timeout tuning

Day 11-12: Agent 6 - Evaluation Agent

Import & Configure:

```
n8n import:workflow --input=n8n/workflows/06_evaluation_agent.json
```

Prerequisites:

- Test set balanced (≥30 videos per class)
- TensorRT engine from training
- DeepStream configured on Jetson

Test:

```
# Trigger evaluation
curl -X POST http://localhost:5678/webhook/agent/evaluation/start \
  -H "Content-Type: application/json" \
  -d '{
    "engine_path": "/opt/reachy/models/emotion_test.engine",
    "test_manifest": "/media/.../test_manifest.jsonl"
  }'
```

Verify:

- SSH to Jetson and check inference logs
- Review confusion matrix in MLflow
- Verify Gate B metrics (latency ≤250ms, F1 ≥0.80)

Success Criteria:

- ✅ Test set inference completes
- ✅ Metrics logged to MLflow
- ✅ Gate B validation enforced

- ✅ evaluation.completed event emitted
- ✅ Test set remains unlabeled (privacy)

Time: 6-8 hours

Phase 4C: Deployment & Operations (Days 12-15)

Day 12-13: Agent 7 - Deployment Agent

Import & Configure:

```
n8n import:workflow --input=n8n/workflows/07_deployment_agent.json
```

Test Shadow Deployment:

```
curl -X POST http://localhost:5678/webhook/agent/deployment/promote \
  -H "Content-Type: application/json" \
  -d '{
    "engine_path": "/workspace/experiments/run_001/emotion.engine",
    "target_stage": "shadow"
  }'
```

Verify:

- Engine copied to Jetson shadow slot
- No traffic routing change
- Health check passes

Test Canary:

- Approve shadow → canary
- Verify 10% traffic routing (if applicable)
- Monitor for 30 minutes
- Check GPU temp, FPS, latency

Test Rollout:

- Approve canary → rollout
- Verify full production deployment
- Verify systemd service restart

Success Criteria:

- ✅ Three-stage deployment works
- ✅ Approval gates enforced
- ✅ Health checks at each stage
- ✅ Rollback capability verified
- ✅ Deployment metadata logged

Time: 6-8 hours

Challenges:

- Jetson network connectivity
- DeepStream service restart timing
- Thermal throttling on sustained load

Day 14: Agent 8 - Privacy Agent

Import & Configure:

```
n8n import:workflow --input=n8n/workflows/08_privacy_agent.json
```

Test TTL Enforcement:

```
# Create old test file
touch -d "30 days ago" /media/.../temp/old_video.mp4

# Trigger privacy agent manually
curl -X POST http://localhost:5678/webhook/privacy/purge
```

```
# Verify old file deleted
```

Test GDPR Deletion:

```
curl -X POST http://localhost:5678/webhook/privacy/purge \
  -H "Content-Type: application/json" \
  -d '{
    "video_id": "<UUID>",
    "reason": "user_request"
  }'
```

Verify:

- File deleted from filesystem
- Database updated (split='purged')
- Audit log entry created

Success Criteria:

- ✅ TTL purge works on schedule
- ✅ Manual deletion works
- ✅ Audit trail complete
- ✅ Soft delete in DB (recovery possible)

Time: 3-4 hours

Day 14-15: Agent 9 - Observability Agent

Import & Configure:

```
n8n import:workflow --input=n8n/workflows/09_observability_agent.json
```

Verify Metrics Collection:

```
# Check metrics endpoints accessible
curl http://localhost:5678/metrics
```

```
curl http://10.0.4.130:9101/metrics  # Media Mover
curl http://10.0.4.140:9100/metrics  # Gateway

# Verify database population
psql reachy_local -c "SELECT * FROM obs_samples ORDER BY ts DESC LIMIT 20;"
```

Configure Alerting:

- Set up Slack/email for SLO breaches
- Test error trigger (intentionally fail a workflow)
- Verify incident created

Success Criteria:

- ✅ Metrics collected every 30s
- ✅ Database stores time series
- ✅ Error trigger catches failures
- ✅ SLO watchdog alerts on breaches
- ✅ Ready for Grafana dashboards

Time: 4-5 hours

Phase 5: Production Hardening (Days 16-25)

Days 16-18: Monitoring Infrastructure

- Set up Prometheus scraping
- Create Grafana dashboards
- Configure alertmanager
- Test alert routing

Time: 12-15 hours

Days 19-21: Security Hardening

- Implement JWT authentication
- Configure mTLS between services
- Set up secrets management (HashiCorp Vault)
- API rate limiting
- Security audit

Time: 15-18 hours

Days 22-23: High Availability

- Database replication setup
- Service redundancy
- Load balancer configuration
- Failover testing

Time: 10-12 hours

Days 24-25: Documentation & Final Testing

- Operational runbooks
- API documentation
- Incident response procedures
- End-to-end testing
- Performance validation

Time: 12-15 hours

📊 COMPLETE TIMELINE & RESOURCE ESTIMATE

Phase	Duration	Effort	Priority
Phase 4A: Core Agents (1-4)	8 days	24-32 hours	🔴 Critical

Phase 4B: ML Agents (5-6)	4 days	14-18 hours	🔴 Critical
Phase 4C: Deploy/Ops (7-9)	4 days	13-17 hours	🟡 High
Phase 5: Production	10 days	49-60 hours	🟢 Medium
TOTAL	**26 days**	**100-127 hours**	

With focused full-time work: 3-4 weeks

With part-time (4h/day): 6-8 weeks

🚧 KEY CHALLENGES & MITIGATIONS

Challenge 1: n8n Learning Curve

Impact: Medium

Mitigation:

- Workflows already designed (no creation needed)
- Use n8n UI for visual debugging
- MCP tools available for programmatic management

Challenge 2: Cross-Service Integration

Impact: High

Mitigation:

- Phases 1-3 APIs already tested
- Start with Agent 1 (simplest integration)
- Comprehensive error logging
- Correlation IDs for tracing

Challenge 3: Async Workflow Complexity

Impact: Medium

Mitigation:

- Wait nodes handle long-running operations
- Idempotency keys prevent duplicates
- Database state as source of truth

Challenge 4: Resource Constraints

Impact: Medium

Mitigation:

- TAO training uses separate GPU timeline
- n8n lightweight (< 500MB RAM)
- Stagger agent deployments

Challenge 5: Testing Without Production Data

Impact: Low

Mitigation:

- Use test data from Phase 2
- Manual webhook triggers
- Dry-run modes in critical agents

✅ SUCCESS CRITERIA

Phase 4 Complete When:

- All 9 workflows active in n8n
- End-to-end video flow tested (upload → train → deploy)
- Class balance maintained automatically
- Training triggers on dataset readiness
- Deployment gates enforced (shadow → canary → rollout)
- Metrics collected and visible

- Zero data loss in promotion/purge operations

Phase 5 Complete When:

- 99.9% uptime over 7 days
- API p95 latency < 200ms
- All security scans passing
- Backup/recovery tested
- Grafana dashboards operational
- Incident response runbooks documented

🎯 MY FINAL RECOMMENDATION

Adopt Approach 1: Agent-by-Agent Sequential Implementation

Why:

1. **Lowest Risk**: Your existing APIs (Phases 1-3) support these workflows individually
2. **Early Value**: Core agents (1-3) automate tedious manual tasks immediately
3. **Clear Progress**: Each agent is a deliverable milestone
4. **Debugging Simplicity**: Problems isolated to one agent
5. **Natural Dependencies**: Order follows data flow (ingest → label → promote → train → deploy)

Start Here (Day 1):

```
# 1. Import Agent 1
cd /home/rusty_admin/projects/reachy_08.4.2
n8n import:workflow --input=n8n/workflows/01_ingest_agent.json

# 2. Configure PostgreSQL credential
# 3. Configure Media Mover HTTP auth
# 4. Test webhook
curl -X POST http://localhost:5678/webhook/video_gen_hook \
  -H "X-INGEST-KEY: test" \
```

```
-H "Content-Type: application/json" \
-d '{"source_url": "https://example.com/test.mp4", "label": "happy"}'
```

Defer to Phase 5:

- Comprehensive monitoring (basic metrics in Agent 9 sufficient initially)
- Security hardening (local network already protected)
- High availability (single instance sufficient for development)

Ready to begin? I can help you:

1. Import and configure Agent 1 right now
2. Generate test data for validation
3. Debug any integration issues
4. Proceed through the 15-day agent deployment plan